EECE 2140

Computing Fundamentals for Engineers

System Design Overview

Statistical Analysis and Histogram Generator

A `C++` Data Visualization and Analysis Tool

Team Members: **Krish Dhamala**
Daniel Gutierrez
Hue Gomatam

Instructor: Dr. Fatema Nafa

Semester: Spring 2026

Repository: `Histogram-and-Data-Visualizer`

Contents

1 Project Overview **2**
1.1 Motivation 2
1.2 Project Goals 2

2 Basic Functionalities **3**
2.1 Functionality Summary Table 3
2.2 Detailed Functionality Descriptions 4

3 Object-Oriented Programming (OOP) Design **5**
3.1 Class Relationship Overview 5
3.2 OOP Design Table 5
3.3 Key OOP Requirements 8

4 System Overview — Input, Process, Output **9**
4.1 High-Level Description 9
4.2 IPO Diagram 9

5 Pseudo Code **10**

6 C++ Implementation Highlights **12**

7 Challenges, Solutions & Lessons Learned **13**
7.1 Challenges & Solutions 13
7.2 Lessons Learned 13

8 Limitations & Future Work **14**

9 GitHub Repository Structure **15**

References **15**

A Sample Program Output **16**

1 Project Overview

Project Description

The **Statistical Analysis and Histogram Generator** is a multi-class `C++` program that reads numerical datasets from external `.csv` or `.txt` files, computes statistical summaries, and generates both text-based and graphical visualizations. The project demonstrates core object-oriented programming principles and the appropriate use of data structures (`vector`, `struct`) to solve a realistic engineering data analysis problem.

1.1 Motivation

When parsing through large datasets and making fast calculations, `C++` is the industry standard. Especially in the field of physics, many highly precise calculations need to be made at a high frequency. To remedy this issue, this project creates a system in which users can input dense data files and receive an immediate statistical summary and visual representation of the data. Parsing through files with over 1000 data points by hand is impossible, but automated computation allows for rapid calculation of values, and visual models are more capable of communicating information than raw data alone.

1.2 Project Goals

- Implement a multi-class `C++` program that accurately models datasets from external files as histograms and lineplots, with statistical calculations on the dataset.
- Design 3 classes (`Dataset`, `Histogram`, `Statistics`) with proper encapsulation, object composition, and immutability & abstraction.
- Calculate the dataset's mean, median, mode, std. dev, variance, min, and max. Track histogram range, bin ranges, and frequency.
- Implement guardrails on exported histogram by limiting number of bins to maintain image clarity. Line graph scales logarithmically if values are very small.
- Maintain a well-structured repository with descriptive commits, multiple documented test cases, and a complete README.
- Document design decisions, tradeoffs made during implementation, and lessons learned from debugging the program.

2 Basic Functionalities

2.1 Functionality Summary Table

#	Functionality	Description	Input	Output
F1	**Dataset Loading**	Reads `.csv` or `.txt` files, detects delimiter automatically, skips the header row, and validates each data row.	String file path	`vector<DataPoint>`, int data size
F2	**Statistics Calculator**	Computes standard statistical measures typically found in a data report.	Dataset vector	Double for each stat; `DataPoint` for min/max
F3	**Histogram Generator**	Generates a text-based histogram in the console by calculating bin ranges and frequencies, then printing line by line.	Dataset vector, bin count or size	Text-based histogram in console
F4	**Graph Exporting**	Writes a gnuplot script and calls the system to render a PNG image of either a histogram or line plot.	String file name	PNG image saved to `images/`

2.2 Detailed Functionality Descriptions

F1 — Dataset Loading

Description: The `Dataset` class reads a file at a given path and parses each row into a `DataPoint` struct containing `x` and `y` values. The file extension is detected to assign the correct delimiter (`','` for CSV, `' '` for TXT). The first line is always skipped as a header. Each value is converted from string to double via `stod()`, wrapped in a try-catch to handle out-of-range and invalid values.

Input: `const string& fPath`
Output: `vector<DataPoint>` stored internally; returns `bool` for success
Contribution: Primary data source that all other classes rely on.

F2 — Statistics Calculator

Description: The `Statistics` class accepts the dataset vector on construction and immediately computes and stores the mean. All other statistical methods use the stored mean for efficiency.

Metrics computed:

- Mean, Median, Mode (returns `NaN` if no mode exists)
- Standard Deviation, Variance (population)
- `findMax()`, `findMin()` — return full `DataPoint` (x, y)

Input: `const vector<DataPoint>& inputData`
Output: `double` for each statistic; `DataPoint` for min/max
Contribution: Provides quantitative analysis of the dataset.

F3 — Histogram Generator

Description: The `Histogram` class computes bin ranges and frequencies from the dataset's y values, then prints a vertical text-based histogram to the console. Bin count is capped at 10 when set directly; bin size is capped at a maximum of 50 bins. Floating point comparisons use an epsilon of `1e-9` to prevent missed equivalencies.

Input: `const vector<DataPoint>& inputData`, bin count or size
Output: Text-based histogram printed to console
Contribution: Provides visual distribution summary in the console.

F4 — Graph Exporting

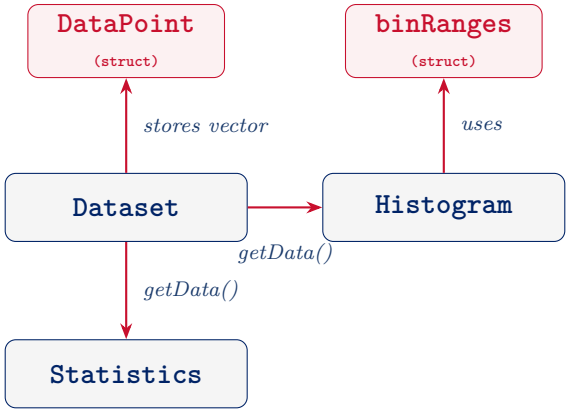
Description: `exportHist()` and `exportLinePlot()` each write a data CSV and a gnuplot `.gp` script to the `images/` folder, then call `system("gnuplot ...")` to render the PNG. The histogram export sets axis ranges, bin widths, and auto-rotates x-axis labels when bin count exceeds 5. The line plot export automatically switches to log scale when `yMin < 1e-6` to handle datasets with very small values.

Input: `const string& fileName`
Output: PNG image saved to `images/histogram.png` or `images/lineplot.png`
Contribution: Provides publication-quality visual output for reports.

3 Object-Oriented Programming (OOP) Design

3.1 Class Relationship Overview

The system consists of **three primary classes** and two supporting structs. Each class encapsulates exactly one aspect of the system to maintain readability and structure.



3.2 OOP Design Table

Class	Attribute / Method	Access	Data Type	Description
`DataPoint`				
(struct)	`x`	**public**	`double`	Independent variable value
	`y`	**public**	`double`	Dependent variable value
`binRanges`				
(struct)	`lowEnd`	**public**	`double`	Lower bound of bin
	`highEnd`	**public**	`double`	Upper bound of bin

Class	Attribute / Method	Access	Data Type	Description
`Dataset`	`data`	**private**	`vector<DataPoint>`	Stores all data as (x,y) pairs
	`filePath`	**private**	`string`	Path of loaded input file
	`dataSize`	**private**	`int`	Count of valid values stored
	`loadFile()`	**public**	`bool`	Opens file, detects delimiter, reads values
	`getData()`	**public**	`const vector<DataPoint>&`	Returns dataset by reference
	`getSize()`	**public**	`int`	Returns number of valid DataPoints
	`clearData()`	**public**	`void`	Resets and empties the dataset

Class	Attribute / Method	Access	Data Type	Description
`Histogram`	`data`	**private**	`vector<DataPoint>`	Dataset passed in from Dataset class
	`range`	**private**	`double`	Range of y values in dataset
	`binNum`	**private**	`size_t`	Number of bins in histogram
	`binSize`	**private**	`double`	Size of each bin
	`getMin()`	**public**	`double`	Gets minimum y value
	`getMax()`	**public**	`double`	Gets maximum y value
	`getRange()`	**public**	`double`	Gets range of y values
	`setBinNums()`	**public**	`void`	Sets bin count (max 10)
	`setBinSize()`	**public**	`void`	Sets bin size (max 50 bins)
	`getBinRanges()`	**public**	`const vector<binRanges>`	Returns computed bin ranges
	`getFrequency()`	**public**	`const vector<int>`	Returns frequency per bin
	`genHist()`	**public**	`void`	Generates text-based histogram
	`exportHist()`	**public**	`bool`	Writes gnuplot script, saves histogram.png
	`exportLinePlot()`	**public**	`bool`	Writes gnuplot script, saves lineplot.png

Class	Attribute / Method	Access	Data Type	Description
`Statistics`	`dataset`	**private**	`vector<DataPoint>`	Stores full dataset
	`mean`	**private**	`double`	Stored mean (computed on construction)
	`compMean()`	**public**	`double`	Calculates and stores mean
	`compMedian()`	**public**	`double`	Calculates median of y values
	`compMode()`	**public**	`double`	Finds most frequent y value (NaN if none)
	`compStdDev()`	**public**	`double`	Calculates standard deviation
	`compVar()`	**public**	`double`	Calculates population variance
	`findMax()`	**public**	`DataPoint`	Returns DataPoint with highest y value
	`findMin()`	**public**	`DataPoint`	Returns DataPoint with lowest y value

3.3 Key OOP Requirements

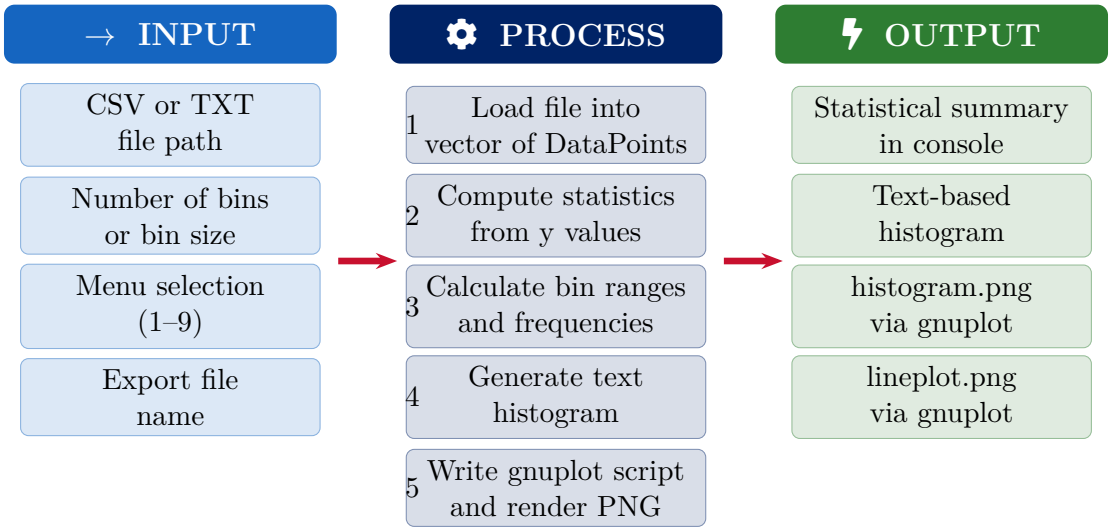
- **3 Classes:** `Dataset`, `Histogram`, `Statistics` — each with a distinct, single responsibility.
- **Private Attributes:** All data members are `private`; external access is only through public methods.
- **Encapsulation:** File parsing, bin calculation, and statistics computation are hidden from the caller.
- **Structs:** `DataPoint` and `binRanges` act as simple public data containers grouping related values together.
- **Pass-by-Reference:** `const vector<DataPoint>&` used consistently in constructors and getters to avoid unnecessary copies.
- **Const Correctness:** All read-only methods marked `const`; only methods that modify state are non-const.
- **Clean Build:** Zero compiler warnings under `-Wall -Wextra -Wpedantic`; zero memory errors verified by Valgrind.

4 System Overview — Input, Process, Output

4.1 High-Level Description

The program follows a classical **Input** → **Process** → **Output** model. All inputs are supplied at runtime through a menu-driven interface. The process executes statistical computations and visualization generation. Outputs are displayed in the console and saved as PNG images.

4.2 IPO Diagram




```
Call chain:   main() → Dataset::loadFile() → Dataset::getData() → Histogram() /
Statistics() → exportHist() / exportLinePlot()
```

5 Pseudo Code

Main Program Loop

```
PROGRAM DataVisualizer
  INITIALIZE dataset, histogram, fileLoaded = false

  LOOP:
    DISPLAY menu options (1-9)
    INPUT userChoice
    VALIDATE userChoice (whole number, 1-9)

    IF userChoice == LOAD:
      CLEAR dataset
      INPUT filePath (getline, supports spaces)
      CALL dataset.loadFile(filePath)
      INITIALIZE histogram(dataset)
      SET fileLoaded = true

    ELSE IF userChoice == STATISTICS:
      INITIALIZE statistics(dataset)
      CALL displayStats(statistics)

    ELSE IF userChoice == GENERATE HISTOGRAM:
      CALL histogram.genHist()

    ELSE IF userChoice == EXPORT HISTOGRAM:
      CALL histogram.exportHist(fileName)

    ELSE IF userChoice == EXPORT LINEPLOT:
      CALL histogram.exportLinePlot(fileName)

    ELSE IF userChoice == EXIT:
      BREAK
  END LOOP
END PROGRAM
```

Dataset::loadFile(filePath)

```
FUNCTION loadFile(filePath):
  OPEN file at filePath
  IF file not open: RETURN false

  DETECT delimiter
    IF .csv: delimiter = ','
    IF .txt: delimiter = ' '
    ELSE: RETURN false

  SKIP header row
  FOR each line in file:
    SPLIT line at delimiter -> xVal, yVal
    REMOVE trailing whitespace
    IF extra columns detected (once): WARN user
    TRY:
      CONVERT xVal, yVal to double
      APPEND DataPoint(x, y) to data
      INCREMENT dataSize
    CATCH out_of_range: SKIP row, WARN user
    CATCH invalid_argument: SKIP row, WARN user
  END FOR

  IF dataSize == 0: RETURN false
  CLOSE file
  RETURN true
END FUNCTION
```

6 C++ Implementation Highlights

Histogram::exportHist() - Key C++ Implementation

```cpp
bool Histogram::exportHist(const string& fileName) {
    if (binNum == 0) return false;

    vector<binRanges> ranges = getBinRanges();
    vector<int> freqs = getFrequency();

    // Write bin data to CSV for gnuplot
    ofstream dataFile("images/" + fileName);
    dataFile << "lowEnd,highEnd,frequency" << endl;
    for (size_t i = 0; i < binNum; i++)
        dataFile << ranges[i].lowEnd << ","
                 << ranges[i].highEnd << ","
                 << freqs[i] << endl;
    dataFile.close();

    // Write gnuplot script
    ofstream gpFile("images/plotHistogram.gp");
    gpFile << "set terminal png" << endl;
    gpFile << "set output 'images/histogram.png'" << endl;
    gpFile << "set boxwidth " << binSize << " absolute" << endl;
    gpFile << "set yrange [0:"
           << *max_element(freqs.begin(), freqs.end()) + 1
           << "] noreverse" << endl;
    // Auto-rotate labels for large bin counts
    if (binNum > 5) {
        gpFile << "set xtics rotate by -45" << endl;
        gpFile << "set bmargin 5" << endl;
    }
    // ... xtics, xrange, plot command ...
    gpFile.close();

    system("gnuplot images/plotHistogram.gp");
    return true;
}
```

Key Design Decisions

D1. `DataPoint` struct for ordered pairs (x,y): Named data members `x` and `y` are more readable than using two separate vectors, improving code clarity throughout all three classes.

D2. Mean stored on construction: `compMean()` is called in the `Statistics` constructor so `compStdDev()` and `compVar()` can use without recomputing.

D3. Epsilon for floating point comparisons: Bin boundary checks use `abs(a - b) < 1e-9` instead of `==` to prevent missed values at exact boundaries.

D4. gnuplot as rendering engine: All data processing and script generation is handled in `C++`; gnuplot is used only for image rendering via `system()`, keeping the visualization logic separate from the computation logic.

7 Challenges, Solutions & Lessons Learned

7.1 Challenges & Solutions

#	Challenge	Solution
1	Extreme value range in waveform data: datasets with y values as small as `1e-309` caused `stod` exceptions and unreadable plots.	Wrapped file reading in try/catch to skip out-of-range values, and added automatic log scale detection when `yMin < 1e-6`.
2	Larger bin counts made x-axis labels overlap in the exported image.	Added a conditional check: if bin count exceeds 5, labels automatically rotate 45 degrees and bottom margin increases to accommodate them.
3	Loading either a `.txt` or `.csv`: different file formats require different delimiters and have different line endings.	Implemented automatic file extension detection to assign the correct delimiter at runtime, and stripped trailing `\r` characters for Windows compatibility.
4	GitHub collaboration conflicts and issues: team members experienced push/pull issues from cloned repositories causing merge conflicts.	Code was shared directly with one team member who committed and pushed on behalf of the team to maintain a clean repository.

7.2 Lessons Learned

- **OOP keeps things clean:** Splitting the project between each class made things far more organized. Each member could work on a file without disrupting the rest of the codebase.
- **GitHub coordination:** Issues concerning repo structure after pushing are probable, such as duplication of files, changes not updating after pushing, and contributor roles not being updated even after making commits.
- **Data structures:** Using vectors and structures, the data was organized in similar ways to how statistical reports and lab documents would present them, making the output intuitive.
- **Debugging process:** Being patient with the iteration process was key. Nothing ever worked on the first try, and relying on fellow team members and online resources was instrumental in resolving issues throughout development.

8 Limitations & Future Work

Current Limitation	Proposed Future Extension
Results are only generated as histogram or line plot.	Integrate additional classes to generate and export other kinds of visualizers (scatter plot, stacked bar chart, etc.).
Bin size is adjustable but with too much data and small enough bin sizes, graph labels extend beyond the image boundary.	Add methods to manipulate the size of the representation itself, or split into multiple graphs/files for easier user readability.
Only able to calculate mean, median, mode, variance, standard deviation, minimum and maximum values.	Add methods to calculate R-value, P-value, linear regression, Z-test, null hypothesis testing, chi-squared tests, etc.
Size of `.png` output is fixed and uncompressed when exported.	Alter `exportHist()` to compress multiple images into one folder for easier sharing and collaboration.
Only one dependent and one independent variable supported per file.	Parse denser files with more columns to explore relationships between one independent and 2+ dependent variables.
Handles small range values but breaks below approximately `10e-350`.	Find alternative methods to convert and represent extremely small range values beyond `double` precision limits.

9 GitHub Repository Structure

Repository Layout

```
Histogram-and-Data-Visualizer/
|
|-- README.md
|-- .gitignore
|-- docs/
|    |-- system_design_overview.pdf
|-- pseudocode/
|    |-- pseudocode.pdf
|-- src/
|    |-- main.cpp
|    |-- Dataset.h / Dataset.cpp
|    |-- Histogram.h / Histogram.cpp
|    |-- StatsCalc.h / StatsCalc.cpp
|-- tests/
|    |-- test.csv / test.txt
|    |-- has_insurance.csv
|    |-- waveform_lab.csv
|    |-- README_tests.md
|    |-- edge_cases/
|        |-- empty.txt, single.csv, identical.csv
|        |-- negative.txt, string.txt
|        |-- three_column.txt, space in fname.csv
|-- images/
     |-- system_diagram.png
     |-- histogram.png
     |-- lineplot.png
```

References

[1] Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley Professional.

[2] Meyers, S. (2014). *Effective Modern C++*. O'Reilly Media.

[3] Cppreference.com. *std::vector*. https://en.cppreference.com/w/cpp/container/vector

A Sample Program Output

Expected Console Output - Test Case (test.csv)

```
Running test case with tests/test.csv..
Data size: 20

--- Statistics ---
Mean: 4.76
Median: 4.6
Mode: 4.2
Std Dev: 1.8621
Variance: 3.4674
Max: (20, 8.5)
Min: (1, 1.5)

Histogram

           x
           x
           x
           x               x
   x       x       x       x       x
   x       x       x       x       x
   x       x       x       x       x
Bin 1   Bin 2   Bin 3   Bin 4   Bin 5

Bin Ranges and Frequencies:
Bin 1: [1.5 - 2.9] Freq: 3
Bin 2: [2.9 - 4.3] Freq: 7
Bin 3: [4.3 - 5.7] Freq: 3
Bin 4: [5.7 - 7.1] Freq: 4
Bin 5: [7.1 - 8.5] Freq: 3
```